

UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



PTI

Informe pràctica 2

Agents

Autor	Gabriel Calvo Catalán, Lucas Diaz Lopez
Grup	10
Curs	2026-2027, quadrimestre de tardor
Data de lliurament	29/09/2026

1. Informació sobre l'ús d'IA

1) Explica com has aplicat la IA per fer la pràctica.

a), b), d), e), h)

He fet servir la IA principalment com a suport per entendre els conceptes nous de la pràctica (RAG, embeddings, protocol MCP, identificadors descentralitzats did:web i credencials verificables). També m'ha ajudat a generar fragments de codi concrets, com ara la nova eina MCP de terratrèmols i l'adaptació de create_index.py per carregar un PDF, a interpretar errors durant l'execució i a polir la redacció final de l'informe.

2) Quines eines has fet servir?

b) Claude. L'he fet servir tant per aclarir conceptes i dubtes de l'enunciat com per revisar codi i millorar la redacció de l'informe.

3) En general quin ordre has fet servir?

b) He començat fent jo una primera proposta i després la IA ho ha completat/polit.

Primer seguia l'enunciat i executava jo mateix cada pas; quan tenia una proposta (de codi o de text) la consultava amb la IA per completar-la, corregir-la o millorar-ne la claredat.

4) En la part de generació/revisió de codi quin mètode has fet servir?

a) He treballat amb l'eina de xat habitual (copiant/enganxant codi del xat als meus fitxers).

Això m'ha permès controlar i entendre cada canvi abans d'aplicar-lo, i fer totes les proves jo mateix al terminal del laboratori.

5) Has fet servir agents? Més d'un simultàniament?

No he fet servir agents de programació autònoms. Paradoxalment, els únics agents de la pràctica són els que he desplegat jo mateix (trust anchor, agent de lloguers, agent RAG i orquestrador), executats en quatre terminals simultàniament.

6) Quant de temps t'has estalviat?

Aproximadament un 30%. L'estalvi més gran ha estat en entendre ràpidament l'estructura de les credencials verificables i dels documents DID, i en resoldre errors de configuració de l'entorn.

7) Penses que has pogut millorar la qualitat de les teves respostes?

Sí. La IA m'ha ajudat a aprofundir en la justificació de les proves (per exemple, per què una modificació de la credencial invalida la signatura) i a presentar els resultats de manera més completa, estructurada i clara.

8) Quin impacte penses que té la IA sobre el teu aprenentatge?

a) Veig un impacte bàsicament positiu, aprenc més de manera més eficient.

Poder preguntar el "per què" de cada pas en el moment en què apareix el dubte fa que entengui millor els conceptes i que dediqui més temps a experimentar que a buscar documentació.

9) Quin ús de la IA recomanaries per fer aquesta pràctica?

Recomanaria fer servir la IA com a tutor: per entendre els conceptes (RAG, MCP, DID, VC) i interpretar la sortida dels terminals, i per generar petites extensions de codi (com una eina MCP nova) que després es revisen i es proven manualment. És important executar i observar cada pas un mateix, ja que l'aprenentatge principal de la pràctica ve de veure com es comporta el sistema.

2. Explicació tècnica del desenvolupament de la pràctica

2.1 Entorn i configuració

La pràctica s'ha realitzat en un equip Ubuntu del laboratori de la FIB amb Python 3.10 i Ollama com a servidor local de models de llenguatge. Cada part té el seu propi entorn virtual (envchatbotcarrental, envchatbot_mcp i envagentslab) per aïllar les dependències.

```
cd chatbot-car_rental
python3 -m venv envchatbotcarrental
source envchatbotcarrental/bin/activate
pip3 install -r requirements.txt
```

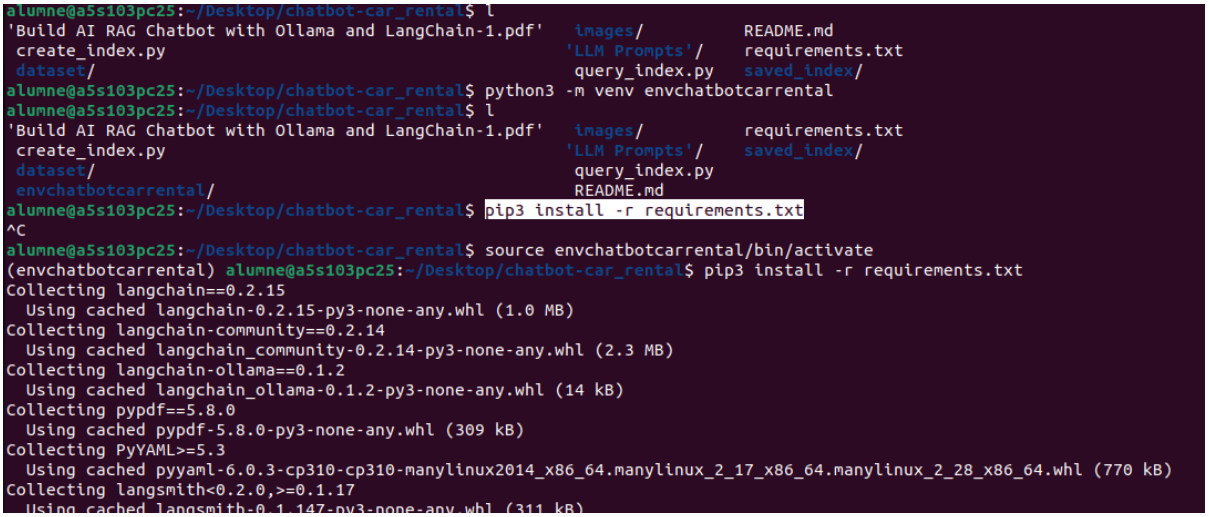


Figura 1. Creació de l'entorn virtual i instal·lació de dependències (LangChain, langchain-ollama, pypdf...).

A continuació es descarreguen els models necessaris amb Ollama: el model de xat (llama3.2 i les variants a comparar) i nomic-embed-text, un model específic d'embeddings.

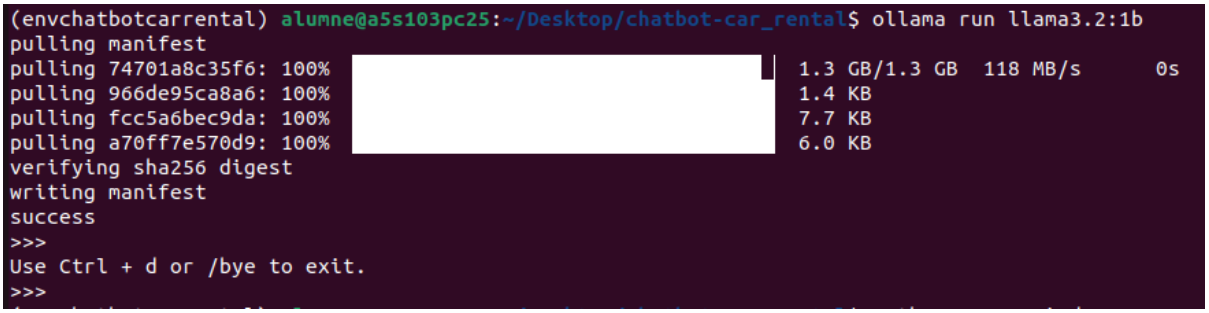


Figura 2. Descàrrega del model llama3.2:1b amb Ollama.

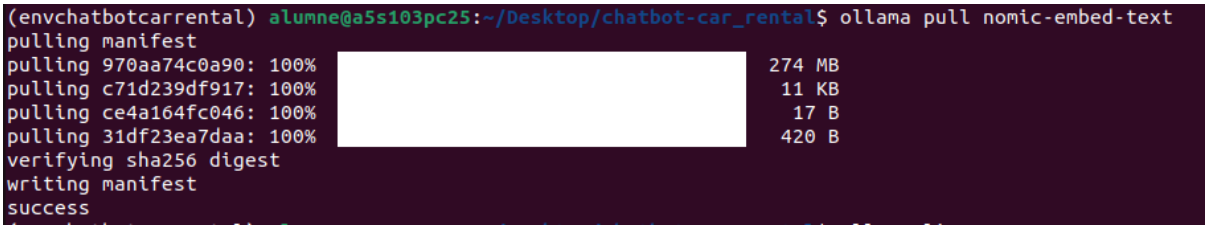


Figura 3. Descàrrega del model d'embeddings nomic-embed-text.

2.2 Part 1 – Chatbot amb RAG

Funcionament

La tècnica RAG permet que el model accedeixi a coneixement extern amb què no s'ha entrenat, en aquest cas dataset/rentals.csv. El script create_index.py divideix el document, en calcula els embeddings amb nomic-embed-text i desa l'índex vectorial a saved_index/vector_index_llama3_2_rentals.pkl. Després, query_index.py carrega l'índex, recupera els fragments més semblants a la pregunta i els passa com a context al LLM.

```
python3 create_index.py   # s'executa una vegada
python3 query_index.py    # xat interactiu
```

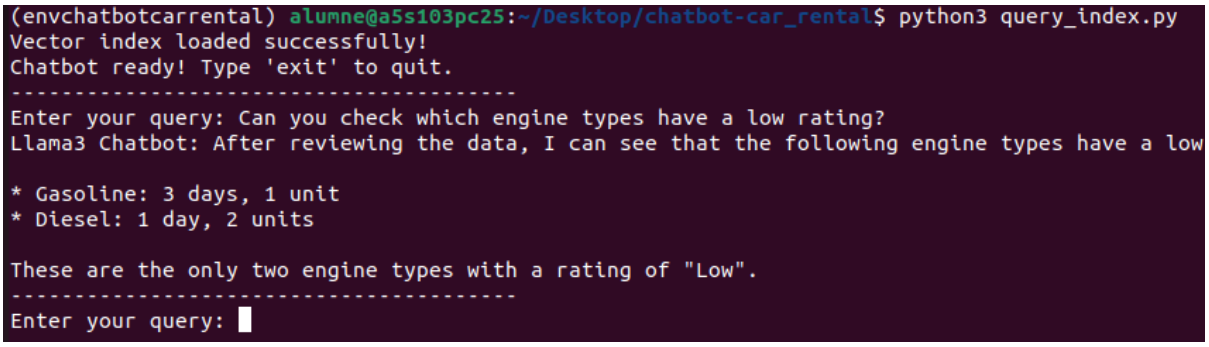


Figura 4. Consulta per defecte: "Can you check which engine types have a low rating?".

El chatbot respon correctament que els tipus de motor amb valoració baixa són Gasoline i Diesel, informació que només pot obtenir del dataset.

Experiment 1 i 2: temps de resposta i qualitat segons el model

Hem afegit a query_index.py la mesura del temps de resposta i hem executat la mateixa sessió (una salutació i la consulta per defecte) amb tres models: llama3.2:1b, llama3.2 (3B) i llama3.1 (8B).

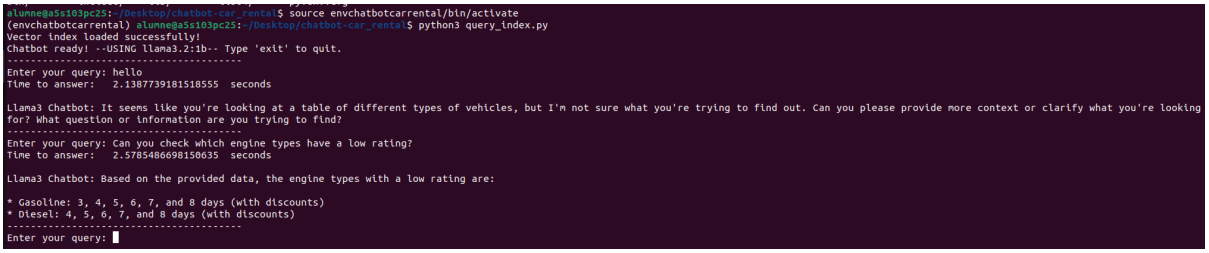


Figura 5. Execució amb llama3.2:1b.

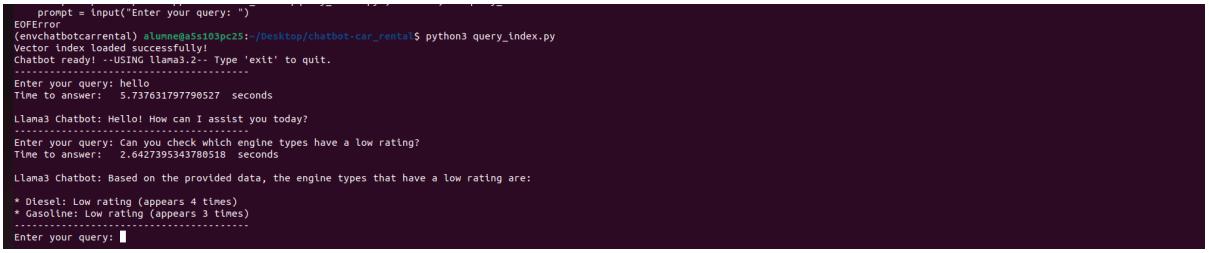


Figura 6. Execució amb llama3.2 (3B).

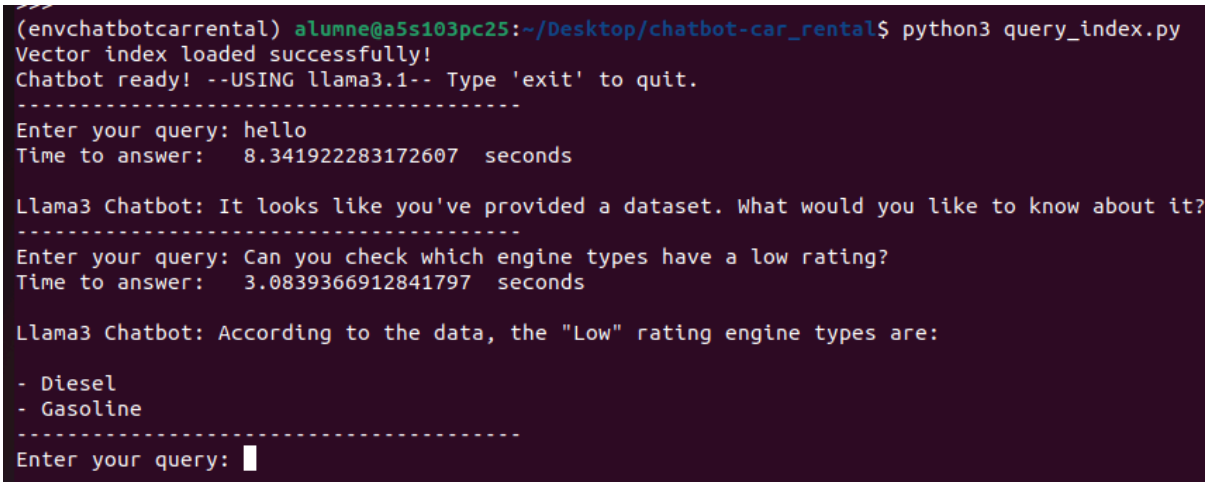


Figura 7. Execució amb llama3.1 (8B).

Model	Temps "hello"	Temps consulta	Qualitat de la resposta
llama3.2:1b	2,14 s	2,58 s	Identifica Gasoline i Diesel, però inventa llistes de dies (al·lucinació parcial) i respon malament a la salutació.
llama3.2 (3B)	5,74 s	2,64 s	Correcta i amb recompte d'aparicions; salutació natural.
llama3.1 (8B)	8,34 s	3,08 s	La més neta i concisa: Diesel i Gasoline, sense detalls inventats.

Conclusió: com més gran és el model, més temps de resposta però millor fidelitat al context. El model de 1B és el més ràpid però tendeix a al·lucinar detalls; llama3.2 ofereix el millor equilibri per al maquinari del laboratori. La primera consulta sempre és més lenta perquè inclou la càrrega del model a memòria.

Experiment 3: dataset en PDF

Hem convertit rentals.csv a un document PDF enriquit (dataset/rentals.pdf) i hem adaptat create_index.py per carregar-lo amb PyPDFLoader en lloc del carregador de CSV:

```
from langchain_community.document_loaders import PyPDFLoader
...
loader = PyPDFLoader("dataset/rentals.pdf")
embeddings = OllamaEmbeddings(model="nomic-embed-text")
```

query_index.py × create_index.py ×

```python
from langchain.indexes import VectorstoreIndexCreator
from langchain_community.document_loaders import PyPDFLoader
from langchain_community.embeddings import OllamaEmbeddings
import pickle
import os
import time

def create_vector_index():
    """Create and save vector index from the dataset"""

    # Create a TextLoader object
    loader = PyPDFLoader("dataset/rentals.pdf")

    # Create an OllamaEmbeddings object
    # NOTE: llama3.2 is a chat model, not an embedding model - asking Ollama
    # to embed with it fails ("This server does not support embeddings").
    # nomic-embed-text is a dedicated embedding model (ollama pull nomic-embed-text).
    embeddings = OllamaEmbeddings(model="nomic-embed-text")

    # Create a VectorstoreIndexCreator object
    index_creator = VectorstoreIndexCreator(embedding=embeddings)

    # Call from_loaders method
    index = index_creator.from_loaders([loader])

    # Save the index to disk
    os.makedirs("saved_index", exist_ok=True)
    with open("saved_index/vector_index_llama3_2_rentals.pkl", "wb") as f:
        pickle.dump(index, f)

    print("Indexing document in vector store completed!")
```

Figura 8. create_index.py adaptat per indexar rentals.pdf.

```
(envchatbotcarrental) alumne@a5s103pc25:~/Desktop/chatbot-car_rental$ python3 create_index.py
/home/alumne/Desktop/chatbot-car_rental/envchatbotcarrental/lib/python3.10/site-packages/langchain/indexes/vectorstore.py:127: UserWarning: Using InMemoryVectorStore as the default vectorstore.This memory
 store won't persist data. You should explicitlyspecify a vectorstore when using VectorstoreIndexCreator
  warnings.warn(
Indexing document in vector store completed!
(envchatbotcarrental) alumne@a5s103pc25:~/Desktop/chatbot-car_rental$ python3 query_index.py
Vector index loaded successfully!
Chatbot ready! --USING llama3.2:1b-- Type 'exit' to quit.
--------------------------------------
Enter your query: Can you check which engine types have a low rating?
Time to answer:   2.2161614894866943  seconds

Llama3 Chatbot: Based on the provided context, the engine types with low ratings are:

* Diesel: Low
* Gasoline: Low
* Diesel: Low
--------------------------------------
Enter your query:
```

Figura 9. Reindexació amb el PDF i consulta amb llama3.2:1b.

El sistema continua responent correctament (Diesel i Gasoline), tot i que amb el model petit apareix un element repetit. Això mostra que RAG és independent del format d'origen: només cal canviar el loader, ja que el PDF es converteix igualment en text i embeddings.

Experiment 4: RAG vs. sense RAG

Hem fet la mateixa pregunta directament al model (ollama run llama3.2) sense context. Sense RAG, el model no coneix el dataset i només pot donar una resposta genèrica o inventada, mentre que amb RAG la resposta es basa en les dades reals. Això demostra el valor principal del RAG: aportar coneixement privat o actualitzat sense reentrenar el model.

Experiment 5: prova pròpia

Com a prova pròpia hem enviat una salutació ("hello") per observar com es comporta el sistema quan la pregunta no té relació amb el dataset. El model de 1B interpreta el context recuperat com si l'usuari li hagués enviat una taula i respon de forma confusa, mentre que els models més grans responen amb naturalitat (Figures 6-8). Això evidencia una limitació del RAG simple: sempre injecta context, encara que la pregunta no ho necessiti.

2.3 Part 2 – Chatbot amb MCP

Amb MCP (Model Context Protocol), el chatbot no rep un índex precalculat sinó que disposa d'eines que pot cridar. El servidor server_mcp.py registra list_rentals (llegeix el mateix rentals.csv) i dues eines meteorològiques d'exemple. El client client_mcp.py connecta Ollama amb el servidor, li informa de les eines disponibles i retransmet les crides.

```
python3 client_mcp.py server_mcp.py
```

```
(envchatbot_mcp) alumne@a5s103pc25:~/Desktop/chatbot_mcp$ python3 client_mcp.py server_mcp.py

Connected to server with tools: ['list_rentals', 'get_alerts', 'get_forecast']

MCP Client Started! (Powered by local Ollama - llama3.2)
Type your queries or 'quit' to exit.

Query:
```

Figura 10. Client MCP connectat amb les tres eines disponibles.

```
(envchatbot_mcp) alumne@a5s103pc25:~/Desktop/chatbot_mcp$ python3 client_mcp.py server_mcp.py

Connected to server with tools: ['list_rentals', 'get_alerts', 'get_forecast']

MCP Client Started! (Powered by local Ollama - llama3.2)
Type your queries or 'quit' to exit.

Query: which rental had the highest number of days?
>>> list_rentals CALLED

[Calling tool list_rentals with args {}]
The rental with the highest number of days is the Diesel engine with 8 days.

Query:
```

Figura 11. La pregunta sobre el lloguer amb més dies provoca la crida a list_rentals.

Es pot veure com el model decideix cridar list_rentals i respon a partir del resultat (el lloguer Diesel de 8 dies). A diferència del RAG, les dades s'obtenen en el moment de la consulta, per tant sempre estan actualitzades.

Extensió pròpia: eina de terratrèmols en temps real

Motivació: volíem demostrar el punt fort de MCP respecte al RAG, que és l'accés a informació viva d'un servei extern. Per això hem afegit una nova eina, get_recent_earthquakes, que consulta l'API pública de l'USGS i retorna els terratrèmols recents que superen una magnitud mínima indicada com a paràmetre.

```python
@mcp.tool()
async def get_recent_earthquakes(min_magnitude: float = 4.5) -> str:
    """Obtiene una lista de terremotos recientes en el mundo que superan
    una magnitud mínima especificada utilizando la API pública del USGS.

    Args:
        min_magnitude: Magnitud mínima (por ejemplo, 4.5 o 5.0).
    """
    url = f"https://earthquake.usgs.gov/fdsnws/event/1/query?format=geojson&minmagnitude={min_magnitude}&limit=5"

    async with httpx.AsyncClient() as client:
        response = await client.get(url)

    if response.status_code != 200:
        return "No se ha podido conectar con el servicio de terremotos."

    data = response.json()
    features = data.get("features", [])

    if not features:
        return "No se han encontrado terremotos recientes con esa magnitud."

    earthquakes = []
    for quake in features:
        props = quake["properties"]
        place = props.get("place", "Ubicación desconocida")
        mag = props.get("mag")

        report = f"""
Ubicación: {place}
Magnitud: {mag} Richter
"""
        earthquakes.append(report.strip())

    return "\n---\n".join(earthquakes)
```

Figura 12. Implementació de l'eina get_recent_earthquakes al servidor MCP.

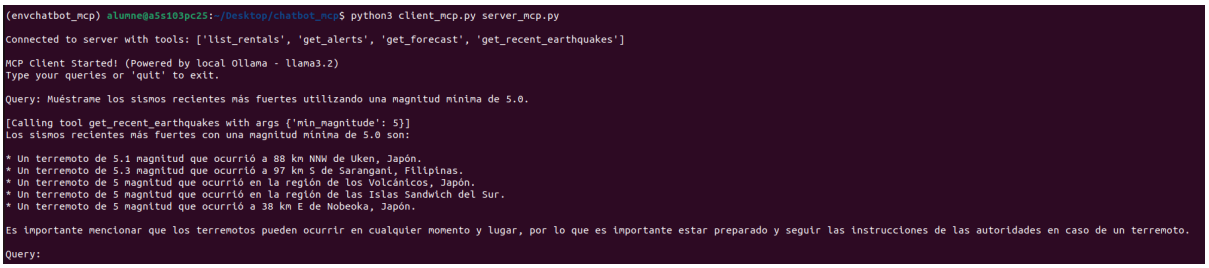


Figura 13. El model crida l'eina amb min_magnitude = 5 i resumeix els resultats.

Resultats: el client detecta automàticament la nova eina (ara n'hi ha quatre). Davant la pregunta en castellà, el model tria l'eina correcta, extreu el paràmetre min_magnitude=5 del text i genera un resum amb dades reals. Només ha calgut afegir una funció amb el decorador @mcp.tool(), sense tocar el client, cosa que demostra l'extensibilitat del protocol.

2.4 Part 3 – Xarxa d'agents de confiança

En aquesta part cada capacitat es converteix en un agent independent, amb el seu port, la seva identitat DID i una credencial que indica què té permès fer.

Procés	Port	Rol
trust_anchor.py	8100	Autoritat de confiança; serveix el seu did.json
agent_rental.py	8101	Ofereix list_rentals
agent_rag.py	8102	Ofereix rag_query amb l'índex de la part 1
agent_orchestrator.py	8103	Punt d'entrada: descobreix, verifica i encamina

3.1 Identitats

Cada agent genera (o carrega) un parell de claus Ed25519 i un identificador did:web mitjançant common/identity.py. Engeguem el trust anchor:



Figura 14. Trust anchor iniciat: mostra el seu DID i la URL on serveix el did.json.



Figura 15. Document DID obtingut amb curl.

El document segueix el format estàndard: un verificationMethod de tipus Ed25519VerificationKey2018 amb la clau pública en format JWK (kty OKP, crv Ed25519), referenciat des d'authentication i assertionMethod. Com a detall interessant, també inclou un servei de revocació (RevocationBitmap2022).

Workarounds per a l'entorn local: (1) el DID codifica el port com a %3A (localhost%3A8100) i la versió de pyvckit no el descodifica en construir la URL de resolució, per això identity.py fa un patch en temps d'importació; (2) la resolució no utilitza /.well-known/did.json sinó el hash multibase de la clau com a segment de ruta, i did_document_route() genera automàticament la ruta Flask correcta. A més, s'usa http:// en lloc d'https://. Tots dos canvis només tenen sentit en desenvolupament local.

3.2 Agent cards

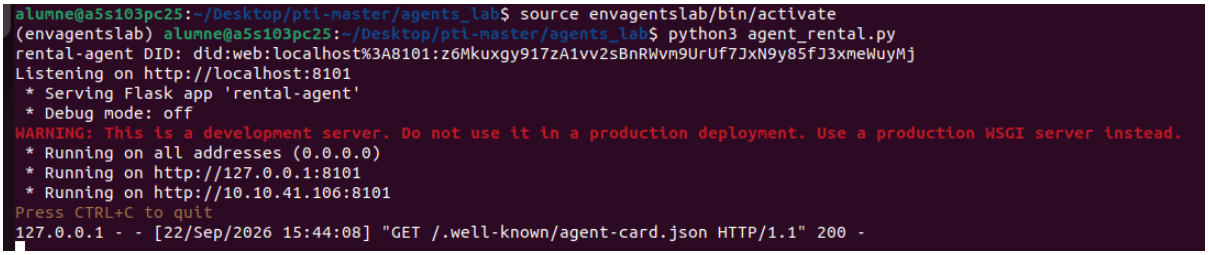


Figura 16. Agent de lloguers iniciat al port 8101 amb el seu DID.

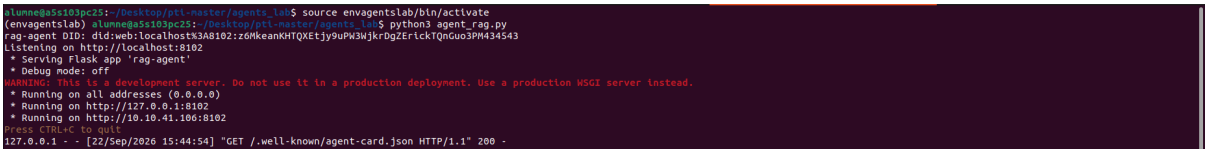


Figura 17. Agent RAG iniciat al port 8102 amb el seu DID.

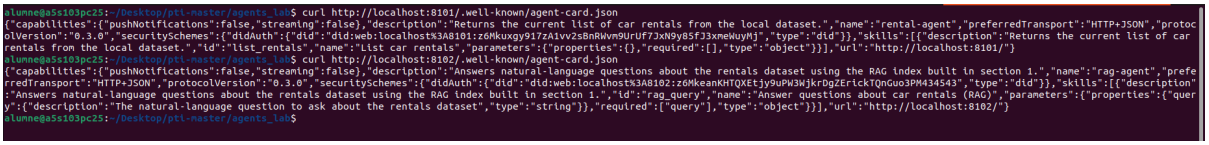


Figura 18. Agent cards obtingudes a /.well-known/agent-card.json.

Cada agent card, amb camps inspirats en el protocol A2A, descriu les skills de l'agent (list_rentals sense paràmetres; rag_query amb un paràmetre query de tipus string definit amb JSON Schema) i, a securitySchemes.didAuth.did, el DID que afirma tenir. La card no està signada, per tant qualsevol servidor podria afirmar qualsevol DID: d'aquí la necessitat de les credencials.

3.3 Emissió de credencials de capacitat

```
python3 issue_credentials.py
```

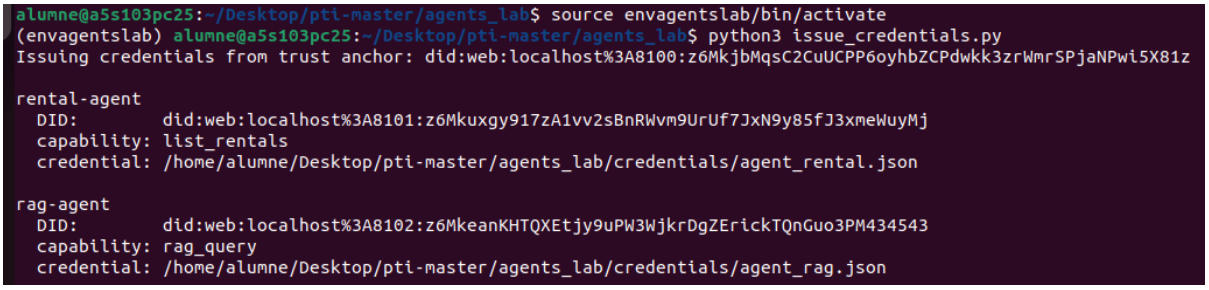


Figura 19. El trust anchor emet una credencial per a cada agent.



Figura 20. L'agent de lloguers serveix la seva credencial a /credential.json.

La credencial és una Verifiable Credential del W3C de tipus AgentCapabilityCredential. Hi podem identificar issuer.id (el DID del trust anchor, port 8100), credentialSubject.id (el DID de l'agent, port 8101), credentialSubject.capabilities (["list_rentals"]) i proof.jws, la signatura Ed25519. Comprovem així que la credencial ha estat emesa pel trust anchor.

3.4 Encaminament amb verificació de confiança

```
python3 agent_orchestrator.py
```

En iniciar-se, l'orquestrador descobreix els dos agents a partir de les seves cards. Abans de cada crida a /invoke, obté la credencial de l'agent i executa verify_capability_credential, que fa quatre

comprovacions: signatura vàlida, emissor de confiança, coincidència del DID del subjecte amb el de la card i capacitat llistada.

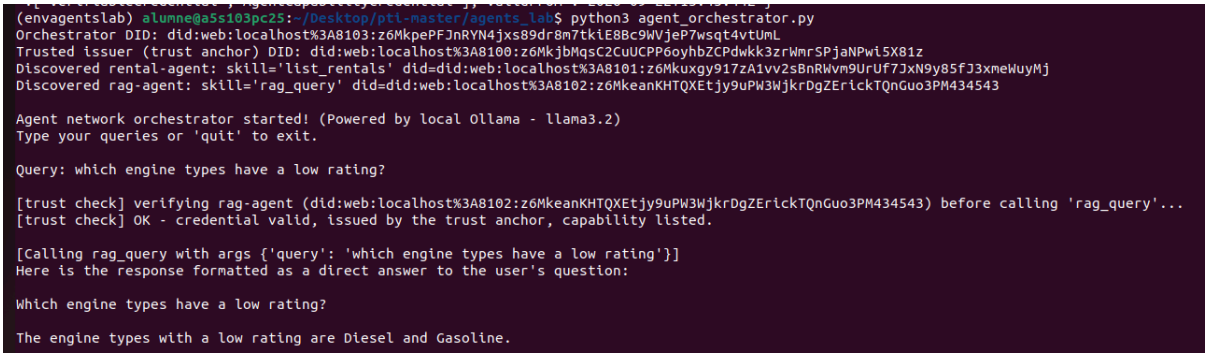


Figura 21. Una pregunta de tipus RAG s'encamina a rag-agent després de superar el trust check.

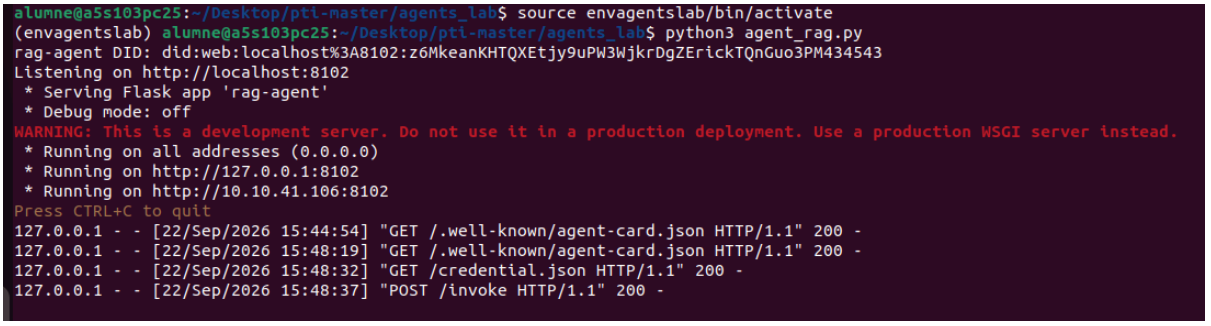


Figura 22. Registre de rag-agent: l'orquestrador consulta la card, la credencial i finalment fa l'/invoke.

Es confirma que la pregunta sobre valoracions s'encamina a agent_rag, que utilitza l'índex de la part 1. El registre mostra l'ordre exacte: primer GET de la credencial i només després POST /invoke.

Prova de manipulació

Hem aturat agent_rental.py, hem modificat a mà credentials/agent_rental.json canviant "list_rentals" per "list_everything" i hem tornat a iniciar l'agent.



Figura 23. Credencial manipulada: la capacitat s'ha canviat a list_everything.

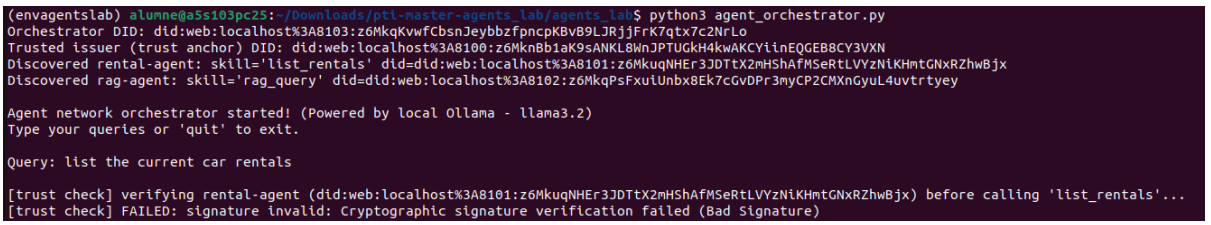


Figura 24. L'orquestrador rebutja la crida: la signatura no és vàlida.

L'orquestrador rebutja la crida amb "signature invalid: Cryptographic signature verification failed (Bad Signature)". La signatura es va calcular sobre el graf RDF normalitzat de la credencial original; en

canviar qualsevol camp, el hash ja no coincideix. Això funciona perquè el context LAB_CONTEXT inclou un @vocab: sense ell, JSON-LD descartaria el camp capabilities en l'expansió i la signatura no el protegiria, de manera que la manipulació passaria desapercebuda.

Propietat de seguretat proporcionada: integritat i autenticitat de les afirmacions de capacitat, i autorització basada en un emissor de confiança. L'orquestrador només invoca un agent si existeix una declaració, signada per l'única autoritat en què confia i no modificada, que diu que aquell DID concret té aquella capacitat concreta. El que encara no garanteix és la possessió de la clau: un servidor maliciós podria servir una còpia exacta d'una credencial vàlida juntament amb la card de l'agent legítim, i aquest és el buit que tanca la prova de possessió de la secció 4.

2.5 Valoració de la solució i de les tecnologies

Aspectes positius

- **RAG:** permet aprofitar dades privades sense reentrenar el model i és independent del format d'origen (CSV o PDF). Tot s'executa en local, sense enviar dades a tercers.
- **MCP:** estandarditza la connexió entre LLM i eines; afegir una capacitat nova (terratrèmols) només requereix una funció al servidor. Dóna accés a dades en temps real.
- **DID i VC:** proporcionen un model de confiança descentralitzat i basat en estàndards W3C, on les autoritzacions es poden verificar criptogràficament sense una base de dades central.

Limitacions trobades

- **Qualitat dels models petits:** el model de 1B al·lucina detalls i tots són lents en maquinari modest (l'orquestrador fa dues crides al LLM per consulta).
- **RAG estàtic:** l'índex és una instantània; si canvien les dades cal reindexar. Per a preguntes d'agregació (màxims, recomptes) MCP és més fiable que la recuperació per similitud.
- **Selecció d'eines:** depèn de la interpretació del LLM, que pot triar una eina incorrecta o afegir text innecessari.
- **Seguretat:** les agent cards no estan signades, no hi ha prova de possessió de la clau, s'usa http:// i cal adaptar la resolució did:web a localhost. També s'hauria de gestionar la revocació i la caducitat de les credencials en un entorn real.

Camps d'aplicació i comparació

RAG és adequat per a assistents sobre documentació interna; MCP, per a assistents que han d'actuar o consultar sistemes vius (APIs, bases de dades). La combinació amb DID i VC és rellevant per a sistemes multiagent entre organitzacions, on cal saber qui és cada agent i què té permès fer. En comparació amb una solució clàssica basada en claus d'API o OAuth amb un servidor central, les VC permeten verificació fora de línia i delegació granular de capacitats, a canvi de més complexitat (JSON-LD, resolució de DIDs, gestió de claus).